



THE KNIFE

MANUALE TECNICO

Università degli Studi dell'Insubria – Laurea Triennale in Informatica

Progetto Laboratorio A

Creato da: Mattia Gerundino, Giorgio Tornaboni, Christian Pianarosa



SOMMARIO

INTRODUZIONE.....	2
STRUTTURA GENERALE DEL SISTEMA DELLE CLASSI.....	2
• CLASSI PRINCIPALI.....	2
• CLASSI UTILI	2
CLASSI PRINCIPALI.....	3
THE KNIFE (MAIN).....	3
RISTORANTE.....	7
UTENTE	9
RECENSIONE	9
UTENTECORRENTE.....	10
CLASSI UTILI	11
FILTRORICERCA	11
GESTORERISTORANTI.....	12
GESTOREUTENTI	13
STRUTTURE DATI	14
SCELTE ALGORITMICHE	15



INTRODUZIONE

THE KNIFE è un progetto realizzato per Laboratorio interdisciplinare A per il corso di laurea in informatica dell'Università degli Studi dell'Insubria anno 2024/2025 è stato sviluppato in Java 22, usando l'ide (es. IntelliJ) per eseguire l'applicazione. Il progetto è stato realizzato e verificato su MacOS.

È una piattaforma che consente di trovare ristoranti in tutto il mondo e selezionarli in base al luogo, alla tipologia del ristorante stesso, alla fascia di prezzo, alla possibilità o meno di prenotare un tavolo o di ordinare da asporto.

STRUTTURA GENERALE DEL SISTEMA DELLE CLASSI

Il progetto è diviso in due parti: le classi principali, che si occupano del controllo dei dati, per ultimo le classi utili.

• CLASSI PRINCIPALI

1. THE KNIFE(MAIN)
2. RISTORANTE
3. UTENTE
4. RECENSIONE
5. UTENTECORRENTE

• CLASSI UTILI

1. FILTRORICERCA
2. GESTORERISTORANTI
3. GESTOREUTENTI



CLASSI PRINCIPALI

THE KNIFE (MAIN)

Classe principale dell'applicazione che gestisce l'interfaccia utente e coordina tutte le operazioni del sistema. Fornisce i menu di navigazione per diversi tipi di utente (guest, cliente, ristoratore) e gestisce il flusso principale dell'applicazione. È progettata per essere eseguita come programma principale, con il metodo main che funge da punto di ingresso. Contiene le diverse strutture dati che tengono in memoria tutti gli oggetti necessari per il corretto funzionamento dell'applicazione.

Componenti Principali:

- Menu principale con opzioni di login, registrazione e accesso guest
- Menu specifici per clienti (ricerca, preferiti, recensioni)
- Menu specifici per ristoratori (gestione ristoranti, risposte alle recensioni)
- Sistema di autenticazione e gestione sessioni utente
- Interfaccia per la ricerca e visualizzazione dei ristoranti

Dipendenze e Utilità:

- java.util.* - Collezioni e strutture dati
- java.io.* - Input/Output per gestione file
- java.time.LocalDate - Gestione date
- java.time.format.DateTimeFormatter - Formattazione date
- java.security.MessageDigest - Crittografia per password
- java.security.NoSuchAlgorithmException - Gestione eccezioni crittografia

Variabili e Costanti della classe:

- static GestoreRistoranti gestoreRistoranti - Gestore per operazioni sui ristoranti
- static GestoreUtenti gestoreUtenti - Gestore per operazioni sugli utenti
- static Scanner scanner - Scanner per input utente
- static UtenteCorrente utenteCorrente - Gestore della sessione utente corrente

GESTORERISTORANTI - Classe che gestisce tutte le operazioni relative ai ristoranti, inclusa la ricerca, il filtraggio, il salvataggio e il caricamento dei dati.

Funzionalità principali:

- Caricamento e salvataggio dati ristoranti
- Ricerca ristoranti con filtri multipli



- Gestione ristoranti per proprietario
- Ricerca per prossimità geografica

GESTOREUTENTI - Classe responsabile della gestione degli utenti, inclusa l'autenticazione, la registrazione e la persistenza dei dati utente.

Funzionalità principali:

- Autenticazione utenti
- Registrazione nuovi utenti
- Verifica unicità username
- Caricamento e salvataggio dati utenti

UTENTECORRENTE - Classe che mantiene lo stato della sessione utente corrente, gestendo il login/logout e l'accesso alle informazioni dell'utente autenticato.

Funzionalità principali:

- Gestione stato di autenticazione
- Accesso ai dati dell'utente loggato
- Operazioni di login/logout

UTENTE - Classe modello che rappresenta un utente del sistema con tutti i suoi attributi e comportamenti.

Attributi principali:

- Nome, cognome, username, password
- Data di nascita, luogo di domicilio
- Ruolo (CLIENTE/RISTORATORE)
- Lista preferiti (per clienti)

RISTORANTE - Classe modello che rappresenta un ristorante con tutte le sue caratteristiche e funzionalità.

Attributi principali:

- Nome, nazione, città, indirizzo
- Coordinate geografiche (latitudine, longitudine)
- Fascia di prezzo, tipo di cucina
- Servizi (delivery, prenotazione online)
- Proprietario, lista recensioni

RECENSIONE - Classe che rappresenta una recensione scritta da un cliente per un ristorante.

Attributi principali:



- Username recensore
- Valutazione (stelle 1-5)
- Testo della recensione
- Data pubblicazione
- Risposta del ristoratore (opzionale)

FILTRORICERCA - Classe che incapsula i criteri di ricerca per i ristoranti, permettendo filtri multipli e combinati.

Filtri disponibili:

- Luogo di ricerca
- Tipo di cucina
- Prezzo massimo
- Disponibilità delivery
- Prenotazione online disponibile

TIPOUTENTE - Enumerazione che definisce i tipi di utente del sistema.

Valori:

- CLIENTE - Utente che può cercare ristoranti e scrivere recensioni
- RISTORATORE - Utente che può aggiungere ristoranti e rispondere alle recensioni

FUNZIONALITÀ PRINCIPALI

Sistema di Autenticazione

- Login con username e password
- Registrazione nuovi utenti
- Accesso guest limitato
- Logout e gestione sessioni

Gestione Ristoranti

- Aggiunta nuovi ristoranti (solo ristoratori)
- Ricerca con filtri multipli
- Visualizzazione dettagli ristorante
- Ricerca per prossimità geografica

Sistema Recensioni

- Scrittura recensioni (solo clienti registrati)



- Modifica/eliminazione proprie recensioni
- Risposta alle recensioni (solo proprietari)
- Calcolo media valutazioni

Funzionalità Cliente

- Gestione lista preferiti
- Visualizzazione cronologia recensioni
- Ricerca personalizzata per luogo di domicilio

Funzionalità Ristoratore

- Gestione portfolio ristoranti
- Monitoraggio recensioni ricevute
- Risposta alle recensioni clienti

PERSISTENZA DATI

Caricamento Dati

- Caricamento automatico all'avvio dell'applicazione
- Gestione errori con continuazione su dati vuoti
- Metodi: caricaDati(), gestoreRistoranti.caricaRistoranti(), gestoreUtenti.caricaUtenti()

Salvataggio Dati

- Salvataggio automatico alla chiusura dell'applicazione
- Gestione errori durante il salvataggio
- Metodi: salvaDati(), gestoreRistoranti.salvaRistoranti(), gestoreUtenti.salvaUtenti()

INTERFACCIA UTENTE

Menu Principale

1. Login
2. Registrazione
3. Continua come Guest
4. Esci

Menu Cliente

1. Cerca ristoranti
2. Visualizza ristoranti vicini



3. Visualizza preferiti
4. Visualizza le mie recensioni
5. Logout

Menu Ristoratore

1. Aggiungi ristorante
2. Visualizza i miei ristoranti
3. Visualizza recensioni dei miei ristoranti
4. Rispondi alle recensioni
5. Logout

Menu Guest

1. Cerca ristoranti
2. Visualizza ristoranti vicini
3. Torna al menu principale

GESTIONE ERRORI

Validazione Input

- Controllo formato date (dd/MM/yyyy)
- Validazione range stelle recensioni (1-5)
- Gestione input numerici non validi
- Controllo unicità username

Gestione Eccezioni

- Try-catch per operazioni I/O
- Gestione NumberFormatException
- Controllo esistenza dati prima dell'accesso
- Messaggi di errore user-friendly

RISTORANTE

Classe che rappresenta un ristorante nel sistema, contenente tutte le informazioni necessarie per la gestione e la ricerca dei ristoranti, incluse le recensioni degli utenti.

Gli attributi della classe:

- String id - Identificatore univoco del ristorante
- String nome - Nome del ristorante



- String nazione - Nazione dove si trova il ristorante
- String città - Città dove si trova il ristorante
- String indirizzo - Indirizzo completo del ristorante
- double latitudine - Coordinata geografica (latitudine)
- double longitudine - Coordinata geografica (longitudine)
- double fasciaPrezzo - Fascia di prezzo del ristorante
- boolean delivery - Indica se il ristorante offre servizio di consegna
- boolean prenotazioneOnline - Indica se il ristorante accetta prenotazioni online
- String tipoCucina - Tipo di cucina offerta dal ristorante
- String proprietario - Proprietario del ristorante
- List<Recensione> recensioni - Lista delle recensioni del ristorante

I costruttori:

- Ristorante(String nome, String nazione, String città, String indirizzo, double latitudine, double longitudine, double fasciaPrezzo, boolean delivery, boolean prenotazioneOnline, String tipoCucina) - Costruttore per la creazione di un nuovo ristorante
- Ristorante(String id, String nome, String nazione, String città, String indirizzo, double latitudine, double longitudine, double fasciaPrezzo, boolean delivery, boolean prenotazioneOnline, String tipoCucina) - Costruttore per il caricamento da file

I metodi usati sono:

- void aggiungiRecensione(Recensione recensione) - Aggiunge una recensione al ristorante
- boolean rimuoviRecensione(String username) - Rimuove una recensione di un utente specifico
- Recensione getRecensione(String username) - Recupera la recensione di un utente specifico
- double getMediaStelle() - Calcola la media delle stelle delle recensioni
- int getNumeroRecensioni() - Restituisce il numero totale di recensioni
- String toString() - Converte l'oggetto in stringa per il salvataggio
- Metodi getter e setter per tutti gli attributi

Integrazione con altre classi:

- Contiene una lista di oggetti Recensione
- Può essere associato a un Utente come proprietario
- Utilizzato nelle operazioni di ricerca con FiltroRicerca



UTENTE

Classe che rappresenta un utente del sistema, gestendo le informazioni personali, l'autenticazione e i ristoranti preferiti.

Gli attributi della classe:

- String nome - Nome dell'utente
- String cognome - Cognome dell'utente
- String username - Username univoco per l'accesso
- String password - Password per l'autenticazione
- LocalDate dataNascita - Data di nascita dell'utente
- String luogoDomicilio - Luogo di domicilio dell'utente
- TipoUtente ruolo - Ruolo dell'utente nel sistema
- List<String> preferiti - Lista degli ID dei ristoranti preferiti

I costruttori:

- Utente(String nome, String cognome, String username, String password, LocalDate dataNascita, String luogoDomicilio, TipoUtente ruolo) - Costruttore standard
- Utente(String nome, String cognome, String username, String password, LocalDate dataNascita, String luogoDomicilio, TipoUtente ruolo, boolean passwordGiaCifrata) - Costruttore per caricamento da file

I metodi usati sono:

- boolean verificaPassword(String password) - Verifica la correttezza della password
- void aggiungiPreferito(String idRistorante) - Aggiunge un ristorante ai preferiti
- void rimuoviPreferito(String idRistorante) - Rimuove un ristorante dai preferiti
- String toString() - Converte l'oggetto in stringa per il salvataggio
- static Utente fromString(String line) - Crea un oggetto Utente da una stringa
- Metodi getter e setter per tutti gli attributi

Integrazione con altre classi:

- Referenzia Ristorante tramite la lista dei preferiti
- Utilizzato da UtenteCorrente per gestire la sessione
- Può creare oggetti Recensione

RECENSIONE

Classe che rappresenta una recensione lasciata da un utente per un ristorante, contenente valutazione, commento e possibile risposta del proprietario.



Gli attributi della classe:

- String username - Username dell'utente che ha lasciato la recensione
- int stelle - Valutazione in stelle (1-5)
- String testo - Testo della recensione
- LocalDate data - Data in cui è stata scritta la recensione
- String risposta - Eventuale risposta del proprietario del ristorante

I costruttori:

- Recensione(String username, int stelle, String testo, LocalDate data) - Costruttore per creare una nuova recensione

I metodi usati sono:

- String toString() - Converte l'oggetto in stringa per il salvataggio
- static Recensione fromString(String line) - Crea un oggetto Recensione da una stringa
- Metodi getter e setter per tutti gli attributi

Integrazione con altre classi:

- Appartiene a un Ristorante specifico
- Creata da un Utente specifico
- Utilizzata per calcolare la media delle stelle del ristorante

UTENTECORRENTE

Classe che gestisce lo stato della sessione utente corrente, mantenendo le informazioni dell'utente autenticato durante l'utilizzo dell'applicazione.

Gli attributi della classe:

- Utente utente - Riferimento all'utente attualmente autenticato

I costruttori:

- UtenteCorrente() - Costruttore che inizializza l'utente corrente a null

I metodi usati sono:

- void setUtente(Utente utente) - Imposta l'utente corrente
- Utente getUtente() - Restituisce l'utente corrente
- boolean isLoggato() - Verifica se c'è un utente autenticato
- void logout() - Effettua il logout impostando l'utente a null



Integrazione con altre classi:

- Contiene un riferimento a un oggetto Utente
- Utilizzato in tutta l'applicazione per verificare lo stato di autenticazione
- Gestisce la sessione dell'utente corrente

CLASSI UTILI

FILTRORICERCA

Classe che incapsula i parametri di ricerca per filtrare i ristoranti secondo vari criteri, utilizzata per implementare la funzionalità di ricerca avanzata.

Gli attributi:

- String luogo - Luogo dove cercare i ristoranti
- String tipoCucina - Tipo di cucina desiderata
- Double prezzoMin - Prezzo minimo della fascia di prezzo
- Double prezzoMax - Prezzo massimo della fascia di prezzo
- Boolean delivery - Filtro per ristoranti con servizio di consegna
- Boolean prenotazioneOnline - Filtro per ristoranti con prenotazione online
- Double mediaMinStelle - Valutazione minima in stelle richiesta

I costruttori:

- FiltroRicerca() - Costruttore vuoto che inizializza tutti i filtri a null

I metodi usati sono:

- Metodi getter e setter per tutti gli attributi di filtro
- Non contiene logica di business, è una classe di supporto per il trasferimento dati

Integrazione con altre classi:

- Utilizzato dai servizi di ricerca per filtrare gli oggetti Ristorante
- Può essere popolato dall'interfaccia utente per costruire query di ricerca personalizzate



GESTORERISTORANTI

Classe che si occupa di gestire l'intero ciclo di vita dei dati dei ristoranti, includendo il caricamento da file CSV, la persistenza delle informazioni, la gestione delle recensioni e le operazioni di ricerca e filtraggio. Questa classe funge da layer di accesso ai dati e-business logic per l'applicazione.

Attributi

- **ristoranti** (List<Ristorante>): Lista principale che contiene tutti i ristoranti caricati nel sistema
- **FILE_RISTORANTI** (static final String): Costante che definisce il percorso del file CSV dei ristoranti ("src/data/ristoranti.csv")
- **FILE_RECENSIONI** (static final String): Costante che definisce il percorso del file delle recensioni ("src/data/recensioni.txt")

Metodi Utilizzati

Costruttore

- **GestoreRistoranti()**: Inizializza la lista dei ristoranti come ArrayList vuoto

Metodi di Caricamento e Salvataggio

- **caricaRistoranti()**: Carica i dati dei ristoranti dal file CSV e le relative recensioni
- **caricaRistorantiDaCSV()**: Metodo privato che parsifica il file CSV dei ristoranti gestendo campi con virgolette e virgole
- **caricaRecensioni()**: Carica le recensioni dal file di testo e le associa ai rispettivi ristoranti
- **salvaRistoranti()**: Salva tutti i ristoranti su file CSV
- **salvaRecensioni()**: Salva tutte le recensioni su file di testo

Metodi di Gestione CRUD

- **aggiungiRistorante(Ristorante ristorante)**: Aggiunge un nuovo ristorante alla lista con controllo duplicati
- **rimuoviRistorante(String id)**: Rimuove un ristorante dalla lista tramite ID
- **aggiornaRistorante(Ristorante ristoranteAggiornato)**: Aggiorna un ristorante esistente

Metodi di Ricerca e Filtraggio

- **cercaRistoranti(FiltroRicerca filtro)**: Cerca ristoranti applicando filtri multipli (luogo, cucina, prezzo, servizi, stelle)
- **getRistorantiVicini(String luogo)**: Restituisce ristoranti in una specifica località
- **getRistoranteById(String id)**: Recupera un ristorante specifico tramite ID
- **getRistorantiDi(String proprietario)**: Restituisce tutti i ristoranti di un proprietario specifico
- **getRistorantiConRecensioneDi(String username)**: Trova ristoranti recensiti da un utente specifico



Metodi di Utilità

- **getTuttiRistoranti()**: Restituisce una copia di tutti i ristoranti
- **getNumeroRistoranti()**: Restituisce il numero totale di ristoranti
- **hasRistoranti()**: Verifica se esistono ristoranti nel sistema

Metodi di Supporto Privati

- **parseCSVLine(String line)**: Parsifica una riga CSV gestendo virgolette e virgole interne
- **cleanField(String field)**: Pulisce un campo rimuovendo virgolette e spazi extra
- **convertPrezzoToNumber(String prezzoStr)**: Converte il formato prezzo simbolico (€, €, €, €, €) in valori numerici
- **parseDouble(String str, double defaultValue)**: Parsing sicuro di valori double con valore di default
- **filtraRistorante(Ristorante ristorante, FiltroRicerca filtro)**: Verifica se un ristorante rispetta i criteri di filtro specificati

Funzionalità Principali

1. **Gestione File**: Caricamento automatico da CSV con parsing robusto e salvataggio persistente
2. **Filtraggio Avanzato**: Ricerca multi-criterio per luogo, tipo di cucina, fascia di prezzo, servizi e valutazioni
3. **Gestione Recensioni**: Integrazione completa con il sistema di recensioni utente
4. **Validazione Dati**: Controlli di integrità per evitare duplicati e dati inconsistenti
5. **Operazioni CRUD**: Supporto completo per Create, Read, Update, Delete dei ristoranti

GESTOREUTENTI

Classe che si occupa di tutte le operazioni relative agli utenti, inclusi registrazione, autenticazione, persistenza dei dati su file, e fornisce metodi per la gestione e il recupero delle informazioni degli utenti. La classe implementa un sistema di crittografia per le password e gestisce due tipi di utenti: clienti e ristoratori.

Attributi principali:

- **FILE_UTENTI**: Costante che definisce il percorso del file di persistenza degli utenti ("src/data/utenti.txt")
- **SEPARATORE**: Costante che definisce il separatore utilizzato nel file (;)
- **DATE_FORMATTER**: Formattatore per le date nel formato dd/MM/yyyy
- **utenti**: Lista contenente tutti gli utenti del sistema (List<Utente>)

Metodi principali utilizzati:

Metodi di gestione del ciclo di vita:

- **GestoreUtenti()**: Costruttore che inizializza la lista utenti e crea la directory data se non esiste
- **caricaUtenti()**: Carica gli utenti dal file di persistenza



- **salvaUtenti()**: Salva tutti gli utenti nel file di persistenza

Metodi di autenticazione e registrazione:

- **registraUtente(Utente utente)**: Registra un nuovo utente nel sistema, controllando che lo username non esista già
- **autenticaUtente(String username, String password)**: Autentica un utente verificando username e password criptata
- **esisteUsername(String username)**: Verifica se un username è già in uso nel sistema

Metodi di recupero dati:

- **getUtenteByUsername(String username)**: Ottiene un utente specifico tramite username
- **getTuttiUtenti()**: Restituisce una copia di tutti gli utenti
- **getClienti()**: Restituisce solo gli utenti con ruolo CLIENTE
- **getRistoratori()**: Restituisce solo gli utenti con ruolo RISTORATORE

Metodi di gestione utenti:

- **rimuoviUtente(String username)**: Rimuove un utente dal sistema
- **aggiornaUtente(Utente utente)**: Aggiorna le informazioni di un utente esistente

Metodi di utilità:

- **criptaPassword(String password)**: Cripta una password usando algoritmo SHA-256
- **parseUtenteDaLinea(String linea)**: Converte una linea del file in oggetto Utente
- **formatUtentePerFile(Utente utente)**: Formatta un utente per il salvataggio su file
- **getStatistiche()**: Restituisce statistiche sui tipi di utenti registrati
- **verificaIntegrita()**: Verifica l'integrità dei dati degli utenti
- **pulisciDati()**: Rimuove utenti con informazioni incomplete

STRUTTURE DATI

TheKnife è una piattaforma per la ricerca di ristoranti strutturata secondo il pattern **MVC** (Model-View-Controller):

Componenti Principali:

- **TheKnife** (Main/Controller): Gestisce l'interfaccia utente e il flusso dell'applicazione
- **GestoreRistoranti**: Gestisce la collezione di ristoranti e le operazioni di ricerca
- **GestoreUtenti**: Gestisce la collezione di utenti e l'autenticazione
- **UtenteCorrente**: Mantiene lo stato dell'utente loggato



Entità del Dominio:

- **Utente:** Contiene informazioni personali, credenziali, preferiti e ruolo (Cliente/Ristoratore)
- **Ristorante:** Contiene dati geografici, caratteristiche, recensioni e proprietario
- **Recensione:** Contiene valutazione, testo, data e risposta del ristorante
- **FiltroRicerca:** Oggetto per incapsulare i criteri di ricerca

Organizzazione dei Dati:

- **Liste:** Utilizzate per memorizzare collezioni di ristoranti, utenti e recensioni
- **Associazioni:** Ristorante-Proprietario, Utente-Preferiti, Ristorante-Recensioni

SCELTE ALGORITMICHE

1. Ricerca e Filtraggio

- **Algoritmo:** Ricerca lineare con filtri multipli
- **Complessità:** $O(n)$ dove n è il numero di ristoranti
- **Implementazione:** Scansione sequenziale applicando filtri per tipo cucina, prezzo, servizi

2. Ricerca Geografica

- **Algoritmo:** Confronto per nome città (ricerca per prossimità geografica)
- **Complessità:** $O(n)$ per la ricerca per località
- **Limitazione:** Non implementa calcolo di distanza reale basato su coordinate

3. Autenticazione

- **Algoritmo:** Ricerca lineare nella lista utenti
- **Complessità:** $O(n)$ per ogni tentativo di login
- **Sicurezza:** Hashing delle password tramite MessageDigest

4. Gestione Recensioni

- **Algoritmo:** Ricerca lineare per utente specifico
- **Complessità:** $O(r)$ dove r è il numero di recensioni per ristorante
- **Calcolo media:** Ricalcolo dinamico della media stelle

5. Persistenza Dati

- **Algoritmo:** Serializzazione/deserializzazione completa
- **Complessità:** $O(n)$ per caricamento e salvataggio



- **Approccio:** Caricamento all'avvio, salvataggio alla chiusura

Ottimizzazioni Possibili:

- **HashMap** per ricerche $O(1)$ su ID/username
- **Indici geografici** per ricerche spaziali efficienti
- **Caching** per risultati di ricerca frequenti
- **Lazy loading** per dati non immediatamente necessari

Permette una complessità temporale lineare in cambio di una implementazione diretta e mantenibile.

